

PSD → 能用的 UGUI 界面

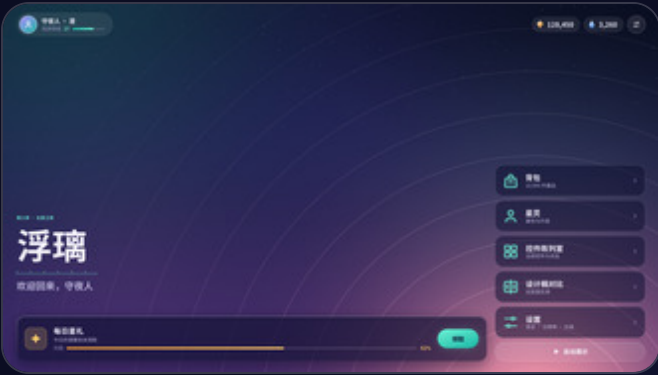
一个把 PSD 设计稿转成 Unity 原生 UGUI 界面资源的工具：Prefab、控件、九宫格切图、给程序用的绑定类，加一份带实测数字的报告；可以在浏览器里预览和修改，修改直接写回 PSD；不按规范画的稿子先由模型整理成标准稿。



Bag.psd 转出的 Prefab，Unity 离屏渲染。517 个图层，4 个按钮、5 个页签、60 个列表条目。



Quest（模拟手绘质感）



Lobby



ConfirmPopup

流程

乱稿 → （模型判断，程序执行）→ 标准 PSD → 检查器 → 切图、界面描述、绑定类 → Unity batchmode 生成 Prefab → 离屏截图 → 与设计稿对比 → 报告。不开任何窗口。

AI 只在一个地方

转换器是确定性的，语义全部来自图层名里的标记。模型只做一件事：判断乱稿里每个图层是什么，写成一份带理由的决定清单；改 PSD、核对结果都由程序做。

制作说明

杨飞洋的个人项目。只用 Unity 原生 UGUI 和 TextMeshPro。完成于 2026 年 10 月 1 日。

交付物

演示视频 psd2ugui.mp4、本文、每张稿子的 HTML 报告（Reports/）、四张示例 PSD（Samples/psd/）。

图层名就是全部输入

规范只管命名和分组。右侧是 Quest.psd 的真实图层清单节选。

名字@btn	按钮。组内同名、不同状态的图层合并成一个 Image
名字@btn@press / @press=0.92	按住时缩小、松开弹回
名字@tabs / @toggle	互斥的页签组（每个子组一个页签）/ 单个开关
名字@list / @grid	列表 / 网格，子组是条目：结构相同、等间距
图层:normal :pressed :disabled :hover	按钮同一图形的不同状态
图层:on :off	页签 / 开关的选中与未选中
图层@9s / @9s=左,上,右,下 / @raw	九宫格：按容差找边距 / 指定边距 / 不切
名字@fx	投影、发光、选中框：不计入节点矩形
名字@anchor=横-纵	指定锚点，不写时自动推断
名字@skip	不导出（参考底图、标注）
名字#someId	给程序用的 id，成为绑定类的字段

```
任务面板@anchor=stretch-middle#panel    <组>
  投影@fx    <图片>   x=276 y=50 1368x1028
  底板@9s    <图片>   x=360 y=110 1200x860
  标题栏    <组>
    标题底@9s    <智能对象>   x=360 y=110 1200x96
    标题#title    <文字>   x=400 y=143 134x32  "每日任务"
    关闭@btn@press#close    <组>
      底    <图片>   x=1480 y=132 52x52
      图标    <图片>   x=1499 y=151 14x14
  分类@tabs#tabs    <组>
    槽    <图片>   x=400 y=230 300x52
    日常#tabDaily    <组>
  ...
任务@list@anchor=stretch-top#quests    <组>
  任务项    <组>
    底@9s    <图片>   x=400 y=306 1120x128
    图标底    <图片>   x=420 y=326 88x88
    图标    <图片>   x=431 y=337 66x64
    标题#title    <文字>   x=530 y=335 86x21  "收集星芒"
    说明#desc    <文字>   x=530 y=365 182x14  "在观测台找到 5 片散落的星芒"
    进度槽@9s    <图片>   x=530 y=395 420x10
    进度条@9s#bar    <图片>   x=530 y=395 420x10
    进度#progress    <文字>   x=962 y=394 28x15  "5 / 5"
    领取@btn@press#claim    <组>
      发光@fx    <图片>   x=1323 y=322 190x96
      底:normal    <图片>   x=1344 y=344 148x52
      底:pressed    <图片> [隐藏]  x=1344 y=344 148x52
      底:disabled    <图片> [隐藏]  x=1344 y=344 148x52
      文字    <文字>   x=1400 y=362 36x18  "领取"
  ...
```

必须在 Photoshop 里处理好的

投影和发光单独成层并标 @fx；文字的字距微调设为 0；图片图层的图层样式要栅格化；只用「正常」混合模式。这些由检查器逐条检查（见后）。

示例稿

四张示例稿由脚本经 Photoshop 生成，含真正的文字图层、蒙版和智能对象。Quest 的底板加了逐像素的颗粒来模拟手绘质感。没有在设计师手画的稿子上测过。

设计稿与引擎截图的像素对比

参照图是 PSD 内嵌的合成图（Photoshop 自己的渲染），引擎图是 Unity 按设计分辨率离屏渲染。任一通道差大于 16/255 记为差异像素。合格线：非文字区域 1%，整图 5%。



设计稿



Unity



差异热力图：差异集中在文字笔画边缘

稿子	图层	图片图层 → 切图	切图像素（万）	按钮 / 页签 / 列表	非文字区域差异	整图差异	文字位置最大偏差
ConfirmPopup	21	12 → 12	264 → 35	3 / 0 / 0	0.000%	0.203%	0.45 px
Lobby	76	39 → 31	289 → 221	3 / 0 / 1	0.000%	0.516%	0.57 px
Bag	517	340 → 109	1013 → 300	4 / 5 / 1	0.023%	0.659%	0.58 px
Quest	94	52 → 43	763 → 408	6 / 3 / 1	0.000%	0.344%	0.58 px

文字是活文字

TextMeshPro，字体、字号、颜色、字距、行距取自 PSD。点文字按基线对齐；段落文字的首行位置 PSD 里没有记录，是量出墨迹后对齐的。

耗时（Bag，本机实测）

读 PSD、切图、九宫格检测 3.8 秒；Unity 启动、生成 Prefab、截图和约 50 张附加渲染 19.1 秒（工程已导入过）。

按钮、页签、列表

PSD 里的状态图层变成原生组件的状态；结构相同、等间距排列的子组变成列表条目，第一个条目存成 Prefab，其余都是它的实例。



页签 1 选中（设计稿的状态）



页签 3 选中：Toggle + ToggleGroup，:on / :off 图层互换



按下：Sprite Swap 换成 :pressed 图层



禁用：换成 :disabled 图层

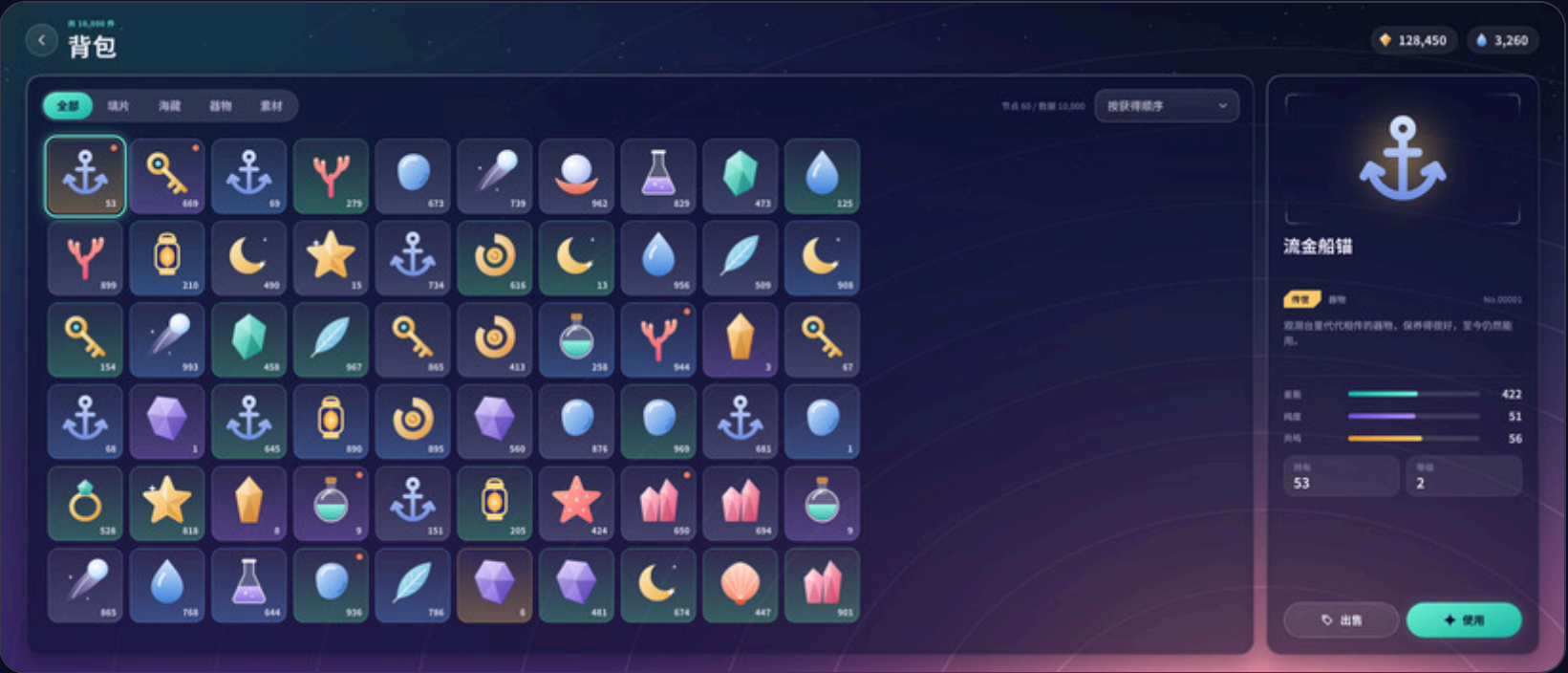
```
Tabs  anchor (0,1)-(0,1) pos 24,-24 size 416x52  ToggleGroup
槽  anchor (0,0)-(1,1) pos 0,0 size 0x0  Image[56x52]
TabAll  anchor (0,0)-(0,1) pos 4,0 size 80x-8  Image(raycast)[no sprite] Toggle[on] ToggleStates
底_On  anchor (0,0)-(1,1) pos 0,0 size 0x0  Image[80x44]
文字_On  anchor (0.5,0.5)-(0.5,0.5) pos 9,0.85 size 54x26.06  TextMeshProUGUI[NotoSansSC-Bold SDF 18]
文字_Off (inactive)  anchor (0.5,0.5)-(0.5,0.5) pos 9,0.85 size 54x26.06  TextMeshProUGUI[NotoSansSC-Bold SDF 18]
TabShards  anchor (0,0)-(0,1) pos 86,0 size 80x-8  Image(raycast)[no sprite] Toggle[off] ToggleStates
底_On (inactive)  anchor (0,0)-(1,1) pos 0,0 size 0x0  Image[80x44]
文字_On (inactive)  anchor (0.5,0.5)-(0.5,0.5) pos 8.5,0.85 size 53x26.06  TextMeshProUGUI[NotoSansSC-Bold SDF 18]
...
文字  anchor (1,0.5)-(1,0.5) pos -43,0.85 size 52x26.06  TextMeshProUGUI[NotoSansSC-Bold SDF 18]
Use  anchor (1,0)-(1,0) pos -28,28 size 186x56  Button[SpriteSwap]
发光  anchor (0,0)-(1,1) pos 0,0 size 42x44  Image[138x100]
底  anchor (0,0)-(1,1) pos 0,0 size 0x0  Image(raycast)[186x56]
图标  anchor (0,0.5)-(0,0.5) pos 60,0 size 20x20  Image[20x20]
文字  anchor (1,0.5)-(1,0.5) pos -43,0.85 size 53x26.06  TextMeshProUGUI[NotoSansSC-Bold SDF 18]
```

```
List  anchor (0,0)-(1,1) pos 0,-26 size -72x-156  ScrollRect
Viewport  anchor (0,0)-(1,1) pos 0,0 size 0x0  RectMask2D Image(raycast)[no sprite]
Content  anchor (0,1)-(1,1) pos 0,0 size 0x780  GridLayoutGroup ContentSizeFitter
格子 <prefab instance>  anchor (0,1)-(0,1) pos 0,0 size 120x120  UIRefs[0 ids]
底  anchor (0,0)-(1,1) pos 0,0 size 0x0  Image[120x120]
图标  anchor (0.5,1)-(0.5,1) pos 0,-17 size 74x72  Image[74x72]
数量  anchor (1,0)-(1,0) pos 3,6.97 size 30x20.27  TextMeshProUGUI[NotoSansSC-Bold SDF 14]
新  anchor (1,1)-(1,1) pos -9,-9 size 10x10  Image[10x10]
选中发光  anchor (0,0)-(1,1) pos 0,0 size 56x56  Image[116x110]
选中框  anchor (0,0)-(1,1) pos 0,0 size 12x12  Image[44x44]
格子 <prefab instance>  anchor (0,1)-(0,1) pos 0,0 size 120x120  UIRefs[0 ids]
...
```

- 格子尺寸、间距、列数从稿子量出：Bag 是 120×120、间距 12、10 列
- 条目各自的图片和文字是实例上的覆盖
- 条目结构不一致、尺寸不一致或排列不等距，检查器报错

锚点推断

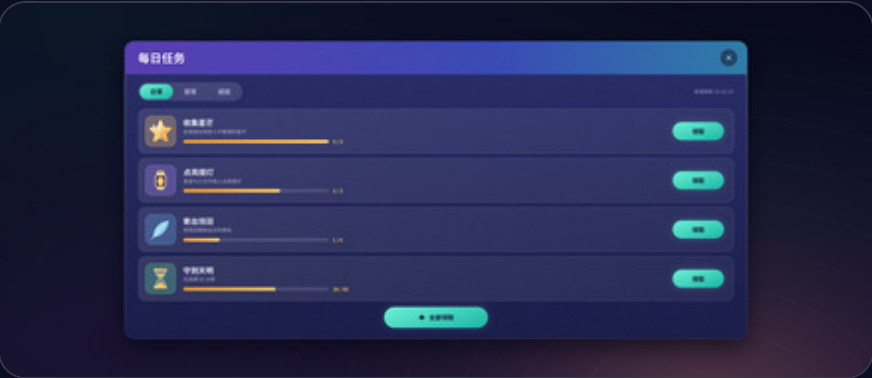
PSD 只有绝对坐标。转换器按节点在父节点里的位置推断锚点：占父节点 80% 以上的拉伸，两侧边距相近的居中，否则贴近的一边。推断不对的用 @anchor 覆盖。



Bag，21:9 画布：列表面板拉宽，详情面板和货币贴右



Bag，4:3 画布：两块面板拉高，按钮贴底



Quest，21:9：面板和四个条目一起变宽，进度文字都紧跟进度条

列表条目另有依据

同一个位置的节点在所有条目里一起看：哪条边到父节点的距离保持不变、另一条边随内容变化，就锚到不变的那条边；看不出来时取多数。逐个条目推断时，Quest 的四个「进度」文字三个判成居中、一个贴右，现在四个一致。

看图发现的问题

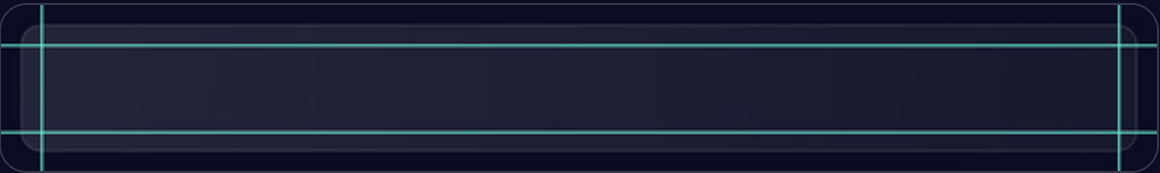
- 网格固定 10 列，宽屏下右边留空
- Bag 的三处锚点是手动标的，自动推断会把列表面板判成贴左
- 设计分辨率之外没有参照图，只能看，不设合格线

九宫格：先找边距，再决定裁不裁

两个问题分开回答。边距在哪：轮廓看不加平滑的 alpha，表面看沿线平滑后的颜色，颗粒被平均掉而圆角不会。中段能不能扔掉：只有各行完全相同才裁小；带渐变或颗粒的中段像素全部保留。



Quest 的底板（带颗粒和渐变）：检测出的边距 28 px，正好是圆角半径。不裁小。



条目底（半透明、带颗粒）：边距 20 px。

稿子	被切的图片图层	按原尺寸拼回的最大误差	拉伸后比较的角
ConfirmPopup	6（1 个 @9s）	2 / 255	没有被拉伸的图
Lobby	14（1 个 @9s）	2 / 255	没有被拉伸的图
Bag	138（2 个 @9s）	2 / 255	24 个，最大差 2 / 255，0 个超线
Quest	31（14 个 @9s）	2 / 255	24 个，最大差 13 / 255，0 个超线

颗粒让第一版失败了

第一版按「相邻两行最大像素差」找中段。Quest 的颗粒让每一行都不同：底板没切出来，条目底的右边距切成了 171 px，拼回误差 18，四角检查报出 4 个角变形。换成现在的做法后边距全部正确。中途还查出一个预乘 alpha 的 16 位整数溢出。

怎么验证

切完按原尺寸拼回，与原图比较；再让 Unity 把九宫格图片单独渲染在设计尺寸、21:9、4:3 三种画布上，比较每张被拉伸的图的四个角。Quest：24 个角，最大差 13 / 255。

绑定类

每个界面一个包装类，每个列表一个条目类。它不是组件，不用挂到 Prefab 上：构造时从 Prefab 的引用表里按 id 取对象。

```
{
  /// <summary>Typed access to the objects of the Quest prefab that carry an id in the PSD.</summary>
  public sealed partial class QuestView
  {
    public readonly GameObject Root;
    /// <summary>#scrim</summary>
    public readonly Image Scrim;
    /// <summary>#panel</summary>
    public readonly RectTransform Panel;
    /// <summary>#title</summary>
    public readonly TMP_Text Title;
    ...
    public QuestView(GameObject root)
    {
      Root = root;
      var refs = root.GetComponent<UIRefs>();
      Scrim = refs.Get<Image>("scrim");
      Panel = refs.Get<RectTransform>("panel");
      ...
    }

    public QuestQuestsItemView[] QuestsItems()
    ...
  }

  /// <summary>One item of the Quests list: an instance of QuestsItem.prefab.</summary>
  public sealed partial class QuestQuestsItemView
  {
    public readonly GameObject Root;
    /// <summary>#title</summary>
    public readonly TMP_Text Title;
    /// <summary>#desc</summary>
    public readonly TMP_Text Desc;
    /// <summary>#bar</summary>
    public readonly Image Bar;
    ...
  }
}
```

类	字段	条目 × 每个条目的字段	没取到对象的
ConfirmPopupView	7	0 × 0	0
LobbyView	16	5 × 0	0
BagView	35	60 × 0	0
QuestView	11	4 × 5	0

同一趟 Unity 里编译并校验

命令行工具在启动 Unity 之前把类写进工程，所以生成 Prefab 时它已经编译好。生成后实际构造一次，逐个字段检查是否取到了对象。

边界

- 只暴露引用，不生成任何行为
- 类是 partial 的，自己的代码写在另一个文件
- 列表条目里的 id 属于条目，登记在条目自己的引用表上

Play 模式：实际点一遍

Unity 以 batchmode 进入 Play 模式，一个虚拟指针通过标准的 StandaloneInputModule 和射线检测去操作 Prefab。红点是指针。不碰桌面，但也不是真人。



Quest：按住「全部领取」。按下态的图，缩到 0.92。



Bag：五个页签逐个点过之后，停在最后点的那一个。



Quest：按住列表拖动，拖了 222 px，内容移动 211 px。

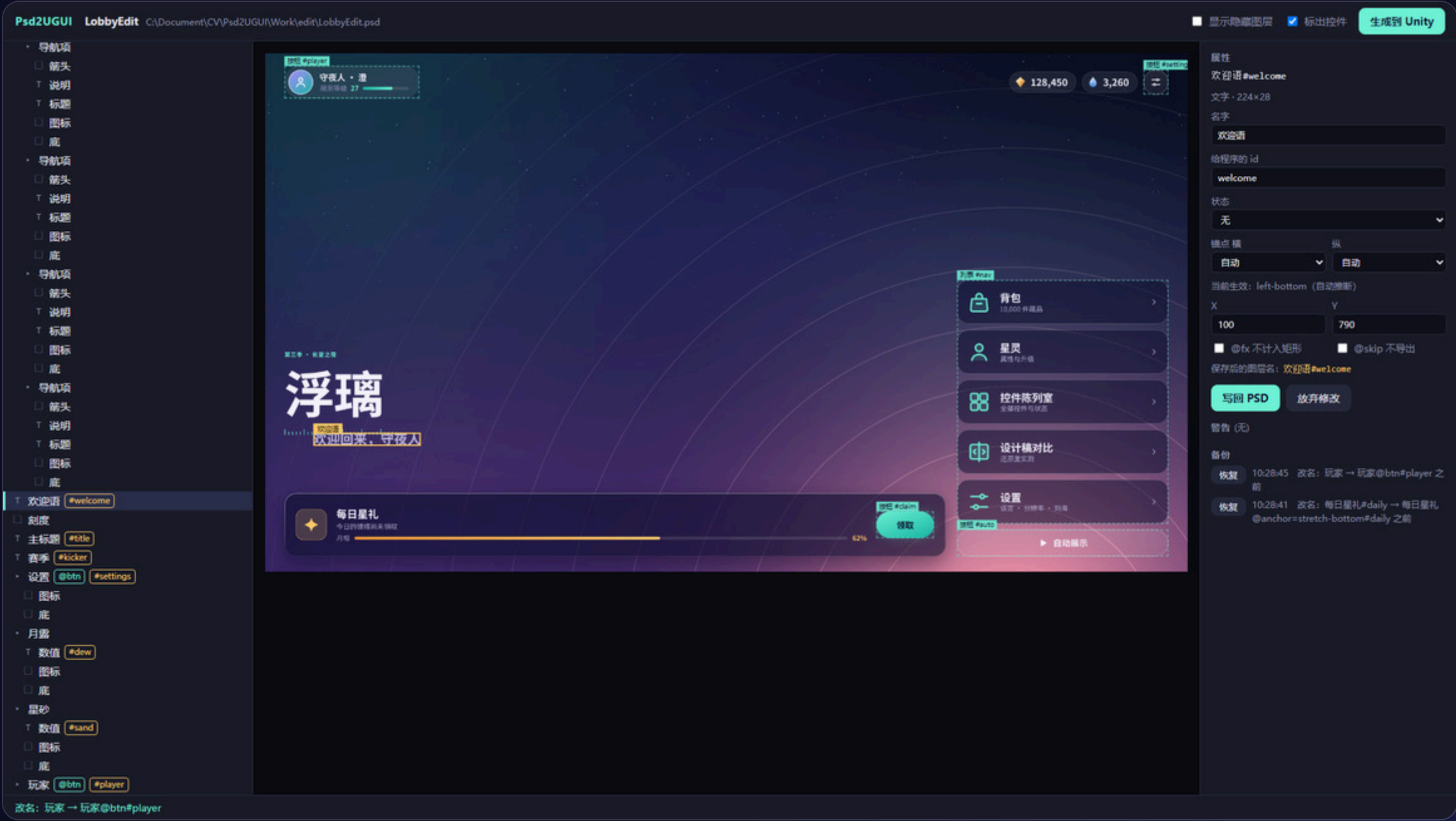
稿子	按钮	页签	列表	检查通过	耗时
ConfirmPopup	3	0	0	7 / 7	6 秒
Lobby	3	0	1	7 / 7	7 秒
Bag	4	5	1	7 / 7	7 秒
Quest	3	3	1	7 / 7	7 秒

检查什么

- 每个按钮收到一次点击；有按下态图层的，按住时显示那张图
- 标了 @press 的缩到设定比例，松开回到原大小；没标的不缩放
- 页签点击后选中，同组其余取消，:on / :off 图层显示正确
- 列表追加条目后按住拖动，内容跟着移动

在浏览器里预览和修改，修改写回 PSD

一个只在本机运行的页面。保存就是写回 PSD：没有旁路文件，用 Photoshop 打开看到的就是最新状态。每次写入前自动备份。



真实页面的截图（无界面浏览器里由脚本操作）：在画面上拖动「欢迎语」，位置栏跟着变，点「写回 PSD」才保存。

页面上改的	写进 PSD 的
名字、给程序的 id	改图层名
控件、状态、@fx、@skip、@press、九宫格	图层名里的标记和后缀
锚点	@anchor 标记；选「自动」则去掉
位置（拖动或填数）	移动图层；文字变换、智能对象位置、蒙版矩形同步更新
尺寸（图片图层）	@size 标记，不重采样像素

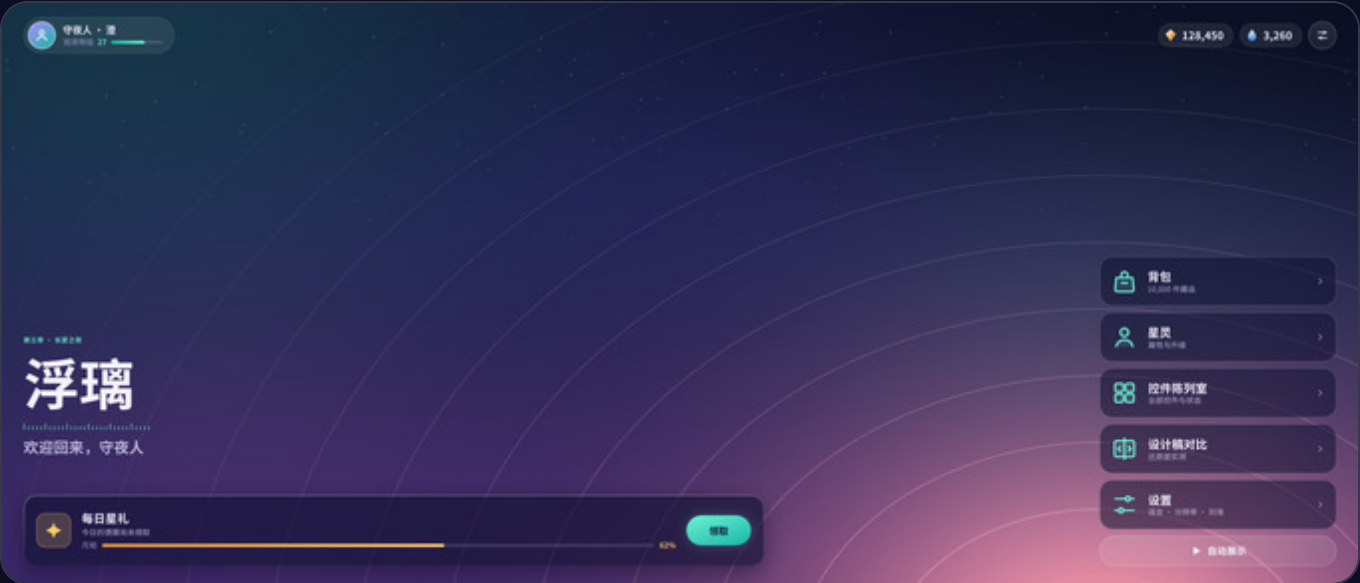
- 不合法的修改被拒绝，文件不动
- 文件被别的程序改过就拒绝写入并重新读取
- 页面上可以直接生成到 Unity 并看到对比结果

页面里改一个锚点

在页面里把「每日星礼」的横向锚点从自动改成 stretch。写进 PSD 的只是图层名里多了 @anchor=stretch-bottom。



点「生成到 Unity」后，结果显示在画面下方：是否合格、差异数字、控件数量、三张图和完整报告的链接。



修改前，21:9 画布：面板保持原宽度



修改后：跟着画布变宽，九宫格让圆角保持不变。面板里的进度条没有跟着变宽，它自己的锚点没改。

直接修改 PSD 是否可靠

把修改写回 PSD 的前提是「程序改过之后 Photoshop 仍然完好打开」。这件事单独验证过，页面的每种修改也都对照文件和 Photoshop 检查过。

程序改 PSD，Photoshop 检查

稿子	Photoshop 读到的图层	文字图层	重新渲染后文字偏差	渲染与预期的差异像素	问题
ConfirmPopup	22	4	0 px	0.009%	0
Lobby	77	24	0 px	0.091%	0

改名、移动整组和单个文字图层、归进新组。对照实验：移动时不同步文字变换、智能对象位置和蒙版矩形，Photoshop 重新渲染后文字跳回原位 42 px，带蒙版的图层消失。

34 / 34

预览服务接口检查

- 每种修改发一遍，再用 psd-tools 读回文件核对
- 恢复最早的备份后，文件与原稿逐字节相同

0.033%

页面写回后的文件：Photoshop 重新渲染 vs 程序合成

- 打开无报错，图层名和顺序一致
- 文字仍是文字图层，智能对象仍是智能对象

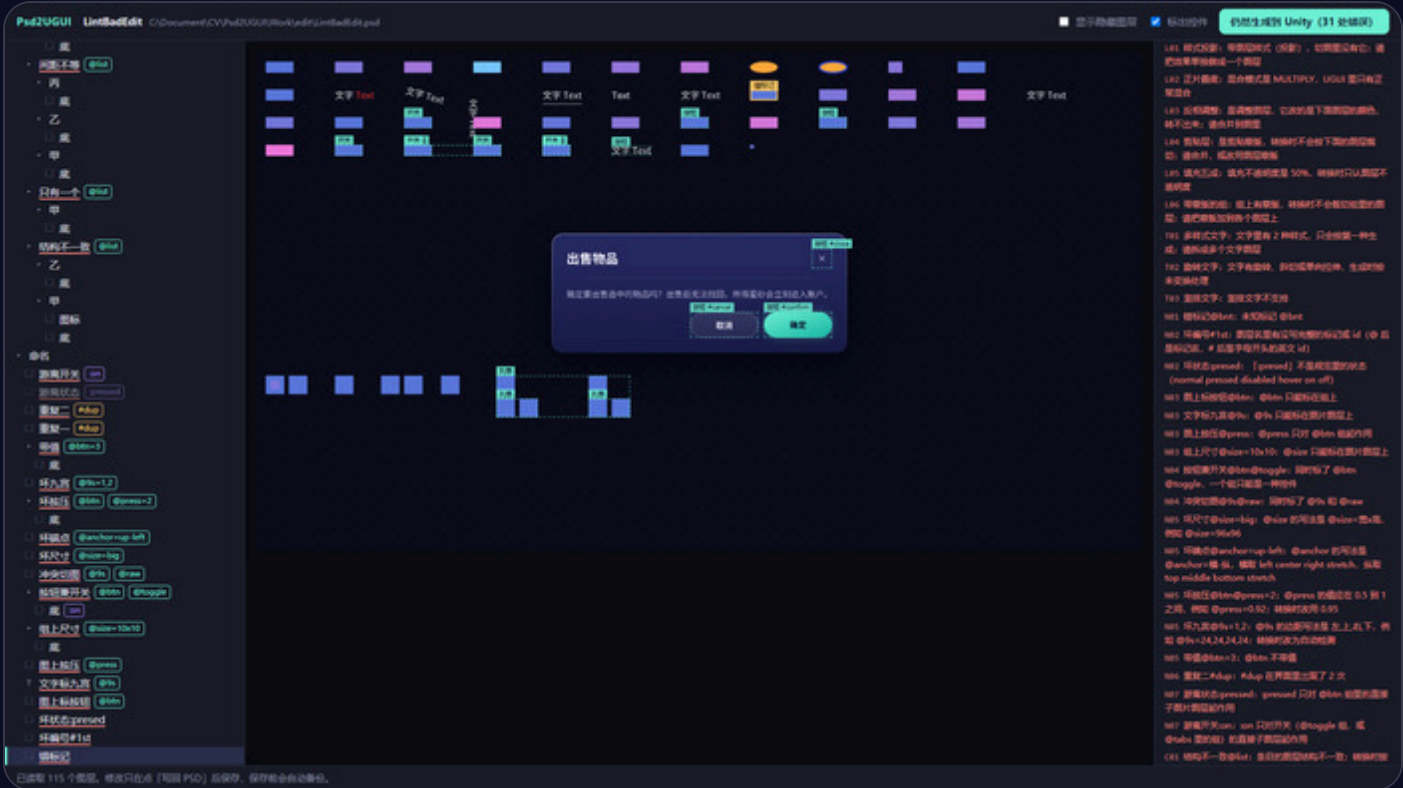
9 / 9

页面检查：无界面浏览器里操作 16 步

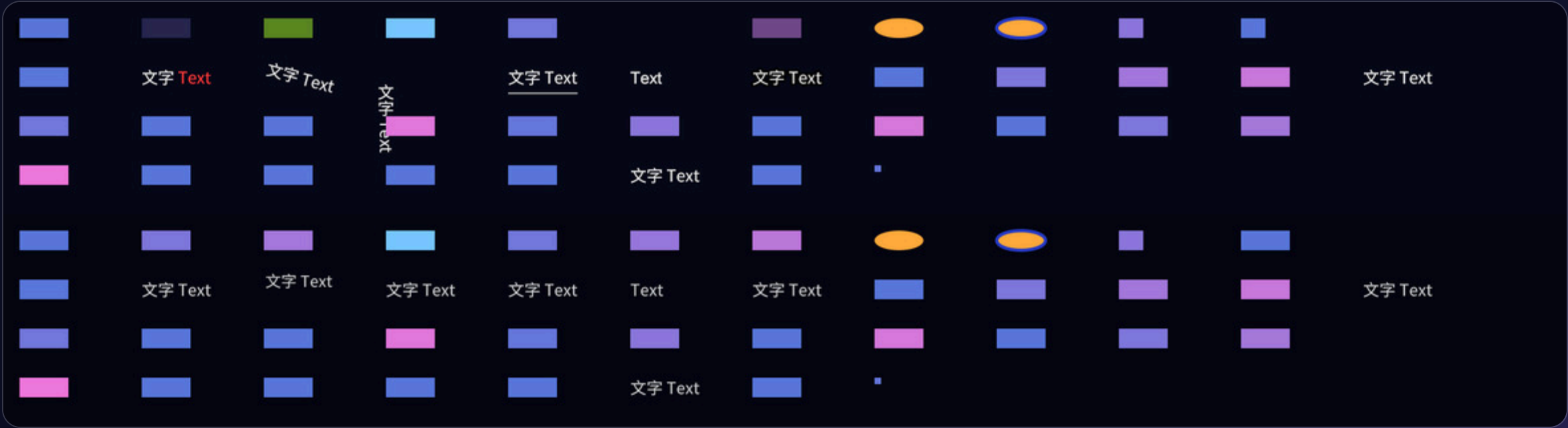
- 点选、填表、拖动、写回、转换、恢复
- 每一步之后核对 PSD 和页面状态

规范检查器：33 条规则

「标准稿」的定义是检查器零错误。错误 = 转出来的画面会和设计稿不一样，或写下的标记无法兑现；有错误就不启动 Unity。每条发现都指到图层。



故意违规的稿子在预览页里：右侧列出全部发现，点一条就选中它所在的图层；名字类的问题可以直接在页面里改。



怎么验证的

- 坏稿：Photoshop 脚本在一张标准稿上加了 94 个图层，每条规则至少一个违规。47 处预期发现全部报出，这些图层上没有多报，原有图层零发现
- 四张标准稿共 708 个图层，零发现
- 规范放行的图层类型（形状、形状描边、纯色填充、矢量蒙版）读出的像素与 Photoshop 的合成图一致。草稿里它们原本被判成错误，比较之后才放行

为什么需要它

把坏稿强行转换，再逐图层和 Photoshop 的画面比：6 个画面类违规里 5 个明显不同（各自范围内 20%–41% 的像素），但整图的判定仍然是「合格」（非文字区域 0.35%）。违规的图层面积小，只看整图数字拦不住。

坏稿强行转换

上半：Photoshop 的画面。下半：Unity。正片叠底、调整图层、剪贴蒙版、填充不透明度、组蒙版、旋转和竖排的文字、文字描边都没有还原；没有违规的对照图层两边一致。

乱稿 → 标准 PSD：模型判断，程序执行

不按规范画的稿子画面是对的，只是程序读不出含义：默认图层名、没有标记、控件没成组。规范化之后它就是一张标准 PSD，和人画的标准稿走同一个检查器、同一个转换器。



模型看到的图（look 命令的输出）：每个图层三个视图，深色底、浅色底、放回原位。隐藏的状态图层放回去才看得出是按下还是禁用。

模型写下的决定（MessyQuest，共 55 条，节选）	它给的理由
2 个图层归组 → 关闭@btn#close	标题栏右上角的圆形底加白色叉号，是关闭按钮
3 个图层归组 → 日常#tabDaily	第一个页签：青色高亮底和两份“日常”文字，当前选中
shadow → 阴影@fx	比面板底大一圈的柔和黑色投影，最多 45% 不透明
panel_bg → 底@9s	面板的圆角深蓝底板，大小随内容变
panel_bg 拷贝 → 多余@skip	隐藏的面板底拷贝，像素与可见的面板底完全相同 (px:d68d)
矩形 1 → 标题栏@9s	面板顶部横贯整宽的紫蓝渐变条
tab → 页签@tabs#tabs	一排三个等宽的格子，日常是选中的那个
矩形 4 → 底:on	选中时才出现的青色高亮底，日常页签里是显示的

模型

- Claude Code skill：导出图层树 → 看图 → 写决定清单
- 不直接改 PSD
- 拿不准的列为「待确认」，写明备选
- 这张稿子整个会话 72 秒

程序

- 只改名和调整分组，不动像素
- 会让重叠图层上下颠倒、或改变显隐和不透明度的操作一律拒绝
- 写完读回：逐图层比像素和文字数据，逐像素比合成画面
- 图层样式、混合模式不处理，列给 Photoshop

盲测：四张乱稿，三轮

乱稿是我从标准稿打乱出来的，我知道答案。所以判断交给全新的子代理：每张稿子一个，只拿到 skill 说明和乱稿，被禁止读标准稿、测试和报告。结果和标准稿逐个控件按矩形比对。

乱稿	图层	怎么乱的	操作	新建的组	模型用时	规范错误 / 警告	转换后：非文字 / 整图差异	Play 检查
MessyPopup	23	分组还在，名字全是默认名	23	0	39 秒	0 / 18 → 0 / 0	0.00% / 0.22%	7 / 7
MessyQuest	88	页签组和按钮组被解散；一个滤色混合、一个图层样式	55	8	72 秒	2 / 57 → 2 / 0	0.85% / 1.18%	7 / 7
MessyLobby	64	完全不分组，导航列表按列排序	59	14	71 秒	0 / 41 → 0 / 0	0.00% / 0.53%	7 / 7
MessyBag	516	默认名，60 个格子是「组 N 拷贝 M」	104	0	125 秒	0 / 417 → 0 / 0	0.02% / 0.66%	7 / 7

命中 / 有偏差 / 漏判 / 误判	标准稿	第一轮	第二轮	第三轮
按钮	16	12 / 4 / 0 / 5	15 / 1 / 0 / 5	15 / 1 / 0 / 5
页签	8	8 / 0 / 0 / 0	8 / 0 / 0 / 0	8 / 0 / 0 / 0
页签组	2	2 / 0 / 0 / 0	2 / 0 / 0 / 0	2 / 0 / 0 / 0
列表	3	3 / 0 / 0 / 0	3 / 0 / 0 / 0	3 / 0 / 0 / 0
@fx 图层	133	133 / 0 / 0 / 1	133 / 0 / 0 / 1	133 / 0 / 0 / 1
@9s 图层	18	6 / 0 / 12 / 8	18 / 0 / 0 / 15	18 / 0 / 0 / 15

- 三轮里程序没有拒绝过任何操作；每张稿子第一次执行就没有命名和结构错误；像素、文字数据、合成画面都与乱稿相同；Photoshop 打开无报错
- 第一轮 的 4 个偏差是同一种：按钮里只有一张隐藏底图时分不清按下态和悬停态
- 误判的 5 个按钮是导航列表的五个条目，模型把条目本身也标成了按钮；多标的 @9s 是进度条和条目底板，对画面没有影响

边界

- 第一轮之后改了工具的看图方式和 skill；其中「单张非灰色隐藏底图判为按下态」是我看过答案后加的
- 乱稿不是别人画的：像素干净、一层一个用途，比真实乱稿好对付
- MessyQuest 剩下的 2 处错误是故意加的图层样式和滤色混合，留给 Photoshop

已知限制

只在四张标准稿和由它们打乱出来的四张乱稿上跑通，就只说在这些稿子上跑通。

还没有的

- 图集
- 进度条、滑条、输入框
- 页面里调整分组和顺序、多选、连续撤销
- 规范化时修像素：图层样式、混合模式只报告，不处理
- 在别人画的乱稿上的验证

做不到一致或有代价的

- 段落文字一行放不下时，Photoshop 挤压标点排进一行，引擎会换行
- 带渐变或颗粒的图不裁小；带颗粒的中段拉伸后成横纹
- @press 连发光一起缩小
- 文字只支持单一样式、横排、无旋转；描边投影没有还原
- 大稿子的页面写回慢：517 个图层一次约 12 秒

验证的范围

- 示例稿由脚本生成，结构比人画的规整；颗粒是均匀随机噪点，不是真实笔触
- 操作由脚本完成：Unity 里是虚拟指针，页面是无界面浏览器。没有覆盖悬停、键盘手柄、触摸，也没有真机
- 只在一台机器上跑过：Unity 2022.3.62f3，Photoshop 2021，Chrome
- 当前报告里的警告共 1 条：文字「观测台里代代相传的器物，保养得很」在引擎里的行数与设计稿不同（高度差 +23.7 px）